

Dreaming of Syscall-less I/O with `io_uring`

Some Assembly Required, Feaver Dreams and Nightmares included

Constantin Pestka¹ und Marcus Paradies²

Abstract:

Over the past decade, storage technology has evolved significantly. Traditional rotational magnetic media, which offered hundreds of IOPS and latencies measured in milliseconds, have been largely replaced by SSDs. Modern SSDs deliver over 10 million IOPS with latencies in the low double-digit microsecond range. This shift has disrupted the assumptions underlying the design of traditional I/O stacks, making them less suitable for modern hardware. To address these challenges, new APIs like NVMe (at the protocol level) and `io_uring` (at the operating system level) have been developed. These APIs aim to eliminate performance bottlenecks that have become apparent as the hardware has advanced. At the OS level, a key focus of these APIs is reducing the number of system calls, as excessive syscalls can severely impact performance on modern systems by disrupting processor caches. Beyond this, many modern APIs share common design principles, whether they operate at the protocol, OS, or user-space library level. They are typically asynchronous, completion-based, and optimized for high-concurrency I/O operations to fully leverage the capabilities of current hardware. In our proposed tutorial, we will begin by introducing `io_uring` and explaining its basic functionality. We will highlight its advantages compared to older I/O APIs and conclude with a discussion of the challenges faced by user-space applications seeking to improve performance with `io_uring`. These include the increased complexity of implementation and scaling issues related to parallelism.

1 `io_uring` – The Basics

The tutorial will begin with an introduction to `io_uring`'s core components such as the **Submission Queue (SQ)** and **Completion Queue (CQ)** and give a high level, conceptual description of how `io_uring` works that will be expanded on in the following sections. We will then explain its basic operational usage from a user perspective based on illustrative examples. This will include a basic introduction to the fundamental challenges and responsibilities of `io_uring` users. These include its asynchronous nature, the responsibility of the user to match the CQEs of completed requests to initially submitted requests, due to out-of-order arrival of completions and the ordering guarantees, or lack thereof, of the execution of requests.

² LMU, Database Systems Group, Oettingenstraße 67, 80538 Munich, Germany, m.paradies@lmu.de

¹ German Aerospace Center, Institute of Data Science, Mälzerstrasse 3, 07745 Jena, Germany, constantin.pestka@c-pestka.de

2 io_uring – The Good

In this section, we will give an overview of io_uring advantages over traditional I/O APIs, such as regular, non-blocking and/or vectored POSIX I/O, AIO, and epoll. To put the magnitude of these advantages into perspective, we will include performance numbers from various sources, such as [Ax19; Di22; HL23; Le18; RT23], as well as our benchmarking results. The main focus here will be on the comparative reduction or complete elision of syscalls during operation. We will also discuss the true asynchronous nature of io_uring, i.e., **guaranteeing to never block in any of its related functions or syscalls the user may execute.** This is in contrast to AIO, which has much softer guarantees, which is detrimental for many applications, as it interferes with proper user space scheduling. Similarly, we will defer the explanation of how this is achieved in io_uring to the next section. We will end this chapter with an overview of the advantages of io_uring. An incomplete list of the relevant features, which we will briefly describe, respectively, is:

1. Registered file descriptors
2. Registered buffers
3. Multi-shot variants of mostly read-style requests, such as `recv`, `read`, `accept`
4. **Zero copy optimizations**
5. Bundled operations (for now only `send`)
6. The ability to cancel in-flight operations
7. Linked requests
8. io_uring exclusive operations e.g. NVMe passthrough, range based `fsync`

3 io_uring – Internals

As a foundation for the evaluation of some important usage patterns and configurations of io_uring in the following, this section will start to introduce some of the internals of io_uring. Examples of misuse of parameters controlling these configurations are plentiful² and often result in high performance degradations. While not the only one, the most important one of these is the configuration of io_uring for fully syscall-less operation, but other parameters, which e.g., allow for the reduction of required *Inter Processor Interrupts (IPIs)*, are also notable.

To explain how these configurations of io_uring work, we begin by explaining what tasks have to be performed within io_uring in general. We will then continue by explaining who does what part of said work at which point in time and explain how that changes based

² See the discussions on the mailing list or `liburing`'s github repository for numerous examples of users not correctly using and understanding these parameters.

on the used configuration. This will most notably include an explanation of how `io_uring` can achieve fully syscall-less operation via SQ polling and how `io_uring` guarantees to never block under any circumstances. We will point out which mechanism is used in which situation to achieve this guarantee to never block, as well as their respective performance impacts. This is likely to be of particular interest to the audience, as this has been frequently misunderstood. Many have assumed that `io_uring` exclusively utilizes the IO WorkQueue, which, ironically, has the highest overhead among the used mechanisms and is hence only used as a last resort. Furthermore, we will include an explanation of how `io_uring` incorporates I/O device polling and explain both the operations of polled and IRQ driven IO in `io_uring`.

4 `io_uring` – The Bad

Based on the details provided in the previous sections, this section we will contrast and discuss the previously listed advantages of `io_uring` with various trade-offs that have to be made when using `io_uring`. The first of these trade-offs to be discussed will be, `io_uring`'s massively increased complexity compared to any other I/O API. `io_uring`'s philosophy of exposing many more kernel-internal tuning knobs to allow for much better performance allows for the optimization opportunities that many high-performance application developers, such as DB developers, have demanded previously. While this is indeed a definitive advantage for such developers, correct usage of these features requires intimate understanding of the inner workings of `io_uring`, but failure to do so will instead result in massive performance regressions or even spurious functional failures, such as deadlocks. This should thus not be underestimated, both in respect to generally increased complexity, but also due to the fact that the documentation for many of these mechanisms and systems is either rather incomplete or even fully absent. To put this point into perspective, we will give some illustrative examples here that can be understood based on the previously explained `io_uring` internals (e.g., related to `IORING_SETUP_COOPTASKRUN` or `IOSQE_ASYNC`).

We will also include a discussion of some additional potential issues for applications, such as the fundamentally asynchronous nature and thus the need for a high quality in application scheduler and extremely limited support in higher-level libraries, such as `libc`, `seastar`, `libuv` or `tokio`. The asynchronous nature specifically either tends to be a complete non-issue for applications or requires major or even full rewrites.

5 `io_uring` – The Ugly

In this section, we will discuss one final difficulty faced by applications when using `io_uring`, which is of particular interest to database systems. This difficulty comes in the form of the large family of issues faced in the integration of `io_uring` in highly parallelized systems, such as storage engines of database systems, general purpose I/O libraries, web servers,

and networking backends of distributed database systems. This discussion will be mostly centered around `io_uring` synchronization methods and their design goals, how these integrate well with some software architectures and notably not so much with others, as well as how certain important features, such as SQ polling are not trivially usable in these scenarios and why that is the case. We will then discuss these more general issues in the context of specific database architectures and touch on some existing integrations, as well as some ongoing efforts and their respective design trade-offs.

6 `io_uring` – The Best We Have Nonetheless?

The final section will conclude with weighting the previously discussed advantages and disadvantages of `io_uring` and compare its utility for application domains such as data bases with some of the available such as the traditional APIs and SPDK, as well as some potential future alternatives such as unikernel approaches.

References

- [Ax19] Axboe, J.: Efficient IO with `io_uring`, [Online; accessed 20-October-2023], 2019, URL: https://kernel.dk/io_uring.pdf.
- [Di22] Didona, D.; Pfefferle, J.; Ioannou, N.; Metzler, B.; Trivedi, A.: Understanding modern storage APIs: a systematic study of `libaio`, SPDK, and `io_uring`. In: Proceedings of the 15th ACM International Conference on Systems and Storage. SYSTOR '22, New York, NY, USA, pp. 120–127, 2022, ISBN: 978-1-4503-9380-5, DOI: 10.1145/3534056.3534945, URL: <https://doi.org/10.1145/3534056.3534945>, visited on: 06/27/2023.
- [HL23] Haas, G.; Leis, V.: What Modern NVMe Storage Can Do, and How to Exploit It: High-Performance I/O for High-Performance Storage Engines. Proc. VLDB Endow. 16(9), pp. 2090–2102, 2023, ISSN: 2150-8097, DOI: 10.14778/3598581.3598584, URL: <https://doi.org/10.14778/3598581.3598584>.
- [Le18] Leis, V.; Haubenschild, M.; Kemper, A.; Neumann, T.: LeanStore: In-Memory Data Management beyond Main Memory. In: 34th IEEE International Conference on Data Engineering, ICDE. Pp. 185–196, 2018, DOI: 10.1109/ICDE.2018.00026, URL: <https://doi.org/10.1109/ICDE.2018.00026>.
- [RT23] Ren, Z.; Trivedi, A.: Performance Characterization of Modern Storage Stacks: POSIX I/O, `Libaio`, SPDK, and `Io_uring`. In: Proceedings of the 3rd Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems. CHEOPS '23, Rome, Italy, pp. 35–45, 2023, ISBN: 9798400700811, DOI: 10.1145/3578353.3589545, URL: <https://doi.org/10.1145/3578353.3589545>.